

Task on CloudFront & Route 53

1. Configure VPC peering in cross regions.

Objective:

To establish private network connectivity between two VPCs located in different AWS regions using VPC Peering.

Steps performed:



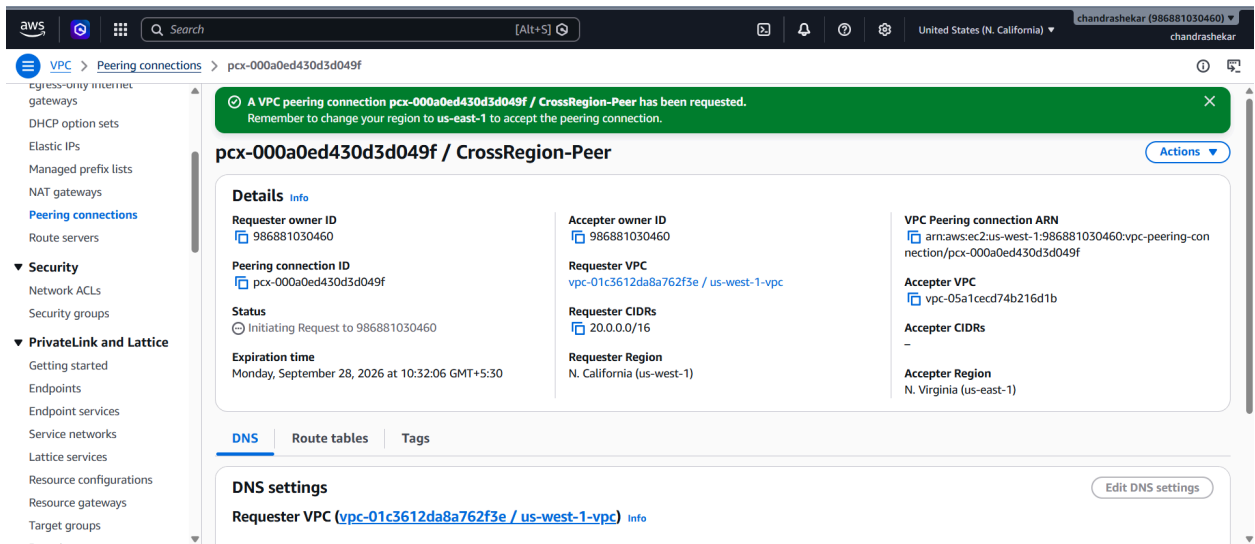
Cross-Region VPC Peering Setup — Step by Step

1. **Create VPCs**
 - Region 1: CIDR `10.0.0.0/16` (us-east-1).
 - Region 2: CIDR `20.0.0.0/16` (us-west-1).
2. **Create Subnets**
 - Public subnet (e.g., `10.0.1.0/24`, `20.0.1.0/24`).
 - Private subnet (e.g., `10.0.2.0/24`, `20.0.2.0/24`).
3. **Attach Internet Gateway**
 - Create IGW in each region.
 - Attach IGW to respective VPCs.
4. **Configure Route Tables**
 - Public RT → `0.0.0.0/0` → IGW.
 - Local RT → VPC CIDR.
 - Add **peering routes**:
 - East RT → `20.0.0.0/16` → Peering connection.
 - West RT → `10.0.0.0/16` → Peering connection.
5. **Create Security Groups**
 - Allow SSH (22) and ICMP from peer VPC CIDR.
 - Allow HTTP (80) if needed.
 - Attach SGs to EC2 instances.
6. **Create VPC Peering Connection**
 - Requester: us-west-1 VPC.
 - Acceptor: us-east-1 VPC.

- Accept request → Status becomes **Active**.
- Launch EC2 Instances**
 - One EC2 in each VPC (Amazon Linux).
 - Assign public IPs for SSH from laptop.
 - Attach correct key pairs (**west.pem**, **east.pem**).
 - Test Connectivity**
 - SSH from laptop → EC2 public IP.
 - From inside EC2, ping peer EC2 private IP.
 - Successful ping = peering works.
 - Optional: SSH across regions using private IP + correct key.
 - Validate Security**
 - Confirm SG rules allow peer CIDRs.
 - (Optional) Enable VPC Flow Logs to monitor traffic.

Outcome

- Two VPCs in different regions are **peered**.
- EC2 instances communicate via **private IPs** across regions.
- Internet access works via IGW.
- Security Groups restrict access properly.



The screenshot shows the AWS Management Console interface for a VPC peering connection. A green notification banner at the top states: "A VPC peering connection pcx-000a0ed430d3d049f / CrossRegion-Peer has been requested. Remember to change your region to us-east-1 to accept the peering connection." Below this, the page title is "pcx-000a0ed430d3d049f / CrossRegion-Peer". The left sidebar shows the navigation menu with "Peering connections" selected. The main content area displays details for the peering connection, including the requester owner ID, requester VPC, requester CIDRs, requester region, and the acceptor VPC. The status is "Initiating Request to 986881030460". The expiration time is "Monday, September 28, 2026 at 10:32:06 GMT+5:30". The DNS settings section is also visible at the bottom.

Details	
Requester owner ID 986881030460	Accepter owner ID 986881030460
Peering connection ID pcx-000a0ed430d3d049f	Requester VPC vpc-01c3612da8a762f3e / us-west-1-vpc
Status Initiating Request to 986881030460	Requester CIDRs 20.0.0.0/16
Expiration time Monday, September 28, 2026 at 10:32:06 GMT+5:30	Requester Region N. California (us-west-1)
VPC Peering connection ARN arn:aws:ec2:us-west-1:986881030460:vpc-peering-connection/pcx-000a0ed430d3d049f	
Accepter VPC vpc-05a1cecd74b216d1b	
Accepter CIDRs -	
Accepter Region N. Virginia (us-east-1)	

DNS settings
Requester VPC (vpc-01c3612da8a762f3e / us-west-1-vpc) Info

ec2-user@ip-20-0-1-241:~

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)

```
$ chmod 400 "west.pem"
```

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)

```
$ ssh -i "west.pem" ec2-user@50.18.65.213
```

ssh: connect to host 50.18.65.213 port 22: Connection timed out

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)

```
$ ssh -i "west.pem" ec2-user@50.18.65.213
```

The authenticity of host '50.18.65.213 (50.18.65.213)' can't be established.

ED25519 key fingerprint is SHA256:83rJV1pw7jr82K10D63jhIPSLA1FBoxBy/Qta+/CNgk.

This key is not known by any other names

Are you sure you want to continue connecting (yes/no/[fingerprint])? yes

Warning: Permanently added '50.18.65.213' (ED25519) to the list of known hosts.

```

#_
~\  #####_      Amazon Linux 2023
~\  #####\
~\  ###|
~\  \#/  _
~\  V~'  '->      https://aws.amazon.com/linux/amazon-linux-2023
~\  .-.-
~\  -/  -/
~\  -/m/'
```

[ec2-user@ip-20-0-1-241 ~]\$ ping 50.18.65.213

PING 50.18.65.213 (50.18.65.213) 56(84) bytes of data.

^Z

[1]+ Stopped ping 50.18.65.213

[ec2-user@ip-20-0-1-241 ~]\$ ping 44.211.230.2

PING 44.211.230.2 (44.211.230.2) 56(84) bytes of data.

^Z

[2]+ Stopped ping 44.211.230.2

[ec2-user@ip-20-0-1-241 ~]\$ ping 20.0.1.241

PING 20.0.1.241 (20.0.1.241) 56(84) bytes of data.

64 bytes from 20.0.1.241: icmp_seq=1 ttl=127 time=0.025 ms

64 bytes from 20.0.1.241: icmp_seq=2 ttl=127 time=0.032 ms

64 bytes from 20.0.1.241: icmp_seq=3 ttl=127 time=0.030 ms

64 bytes from 20.0.1.241: icmp_seq=4 ttl=127 time=0.027 ms

^Z

[3]+ Stopped ping 20.0.1.241

[ec2-user@ip-20-0-1-241 ~]\$ ssh ec2-user@<Private-IP-of-East-EC2>

-bash: syntax error near unexpected token `newline'

[ec2-user@ip-20-0-1-241 ~]\$ ssh ec2-user@20.0.1.241

The authenticity of host '20.0.1.241 (20.0.1.241)' can't be established.

ED25519 key fingerprint is SHA256:83rJV1pw7jr82K10D63jhIPSLA1FBoxBy/Qta+/CNgk.

This key is not known by any other names.

Are you sure you want to continue connecting (yes/no/[fingerprint])? yes

Warning: Permanently added '20.0.1.241' (ED25519) to the list of known hosts.

ec2-user@20.0.1.241: Permission denied (publickey,gssapi-keyex,gssapi-with-mic).

[ec2-user@ip-20-0-1-241 ~]\$ ping 10.0.1.247

PING 10.0.1.247 (10.0.1.247) 56(84) bytes of data.

^Z

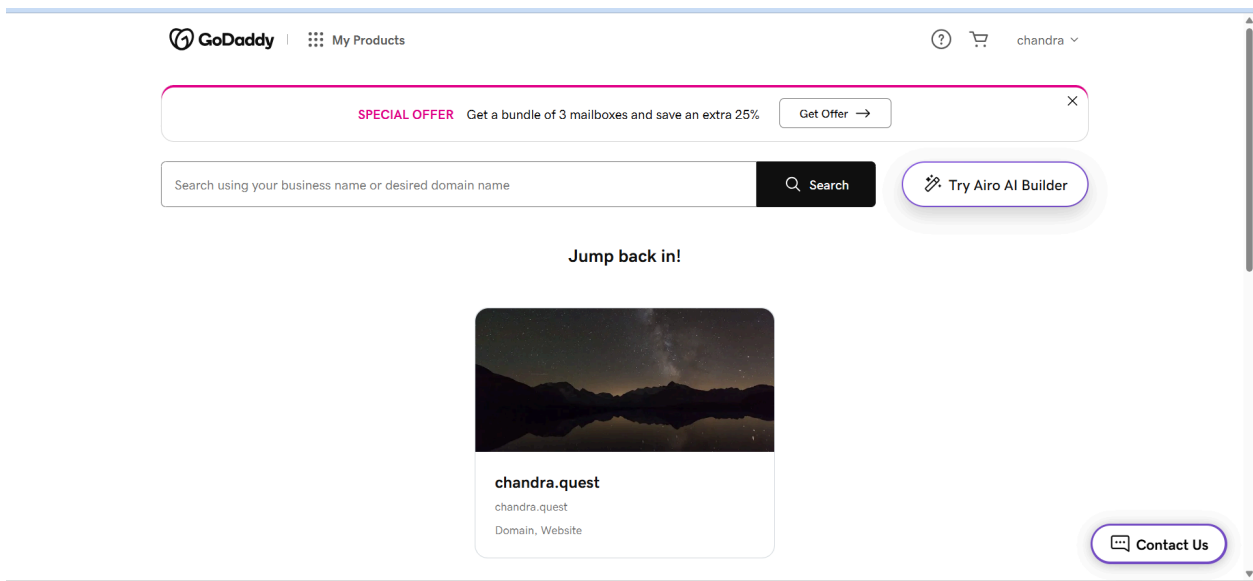
[4]+ Stopped ping 10.0.1.247

[ec2-user@ip-20-0-1-241 ~]\$

4. **Add to Cart**
 - Click “Add to Cart.”
 - Choose registration period (1 year for budget demo).
5. **Add Privacy Protection**
 - GoDaddy provides free domain privacy → keep enabled.
6. **Checkout and Pay**
 - Enter payment details (UPI, card, etc.).
 - Complete purchase.
7. **Verify Domain Ownership**
 - Domain will appear under “My Products.”
 - Example: **chandra.quest** confirmed.
8. **Prepare for AWS Route 53**
 - Later, update DNS nameservers in GoDaddy to point to AWS Route 53 hosted zone.

Outcome

- Domain successfully purchased (**chandra.quest**).
- Cost within budget (₹134.74).



3. Deploy static website in S3.

Task 3 — Deploy Static Website in S3

1. **Bucket Name**
 - Enter a unique name (e.g., `chandra-quest-site`).
 - Region: keep default (us-east-1) or choose nearest.
2. **Object Ownership**
 - Leave **ACLs disabled (recommended)**.
 - This ensures bucket owner enforced.
3. **Block Public Access**
 - **Uncheck “Block all public access.”**
 - This is required for a public website.
 - Confirm by ticking the warning checkbox (“I acknowledge…”).
4. **Bucket Versioning**
 - Keep **Disabled** (not needed for static site).
5. **Tags** (optional)
 - You can skip or add tags for organization.
6. **Create Bucket**
 - Scroll down → click **Create bucket**.
7. **Upload Website Files**
 - Upload `index.html` (and CSS/JS if needed).
8. **Enable Static Website Hosting**
 - Go to **Properties** → Static website hosting → Enable.
 - Set `index.html` as the index document.
9. **Set Bucket Policy**

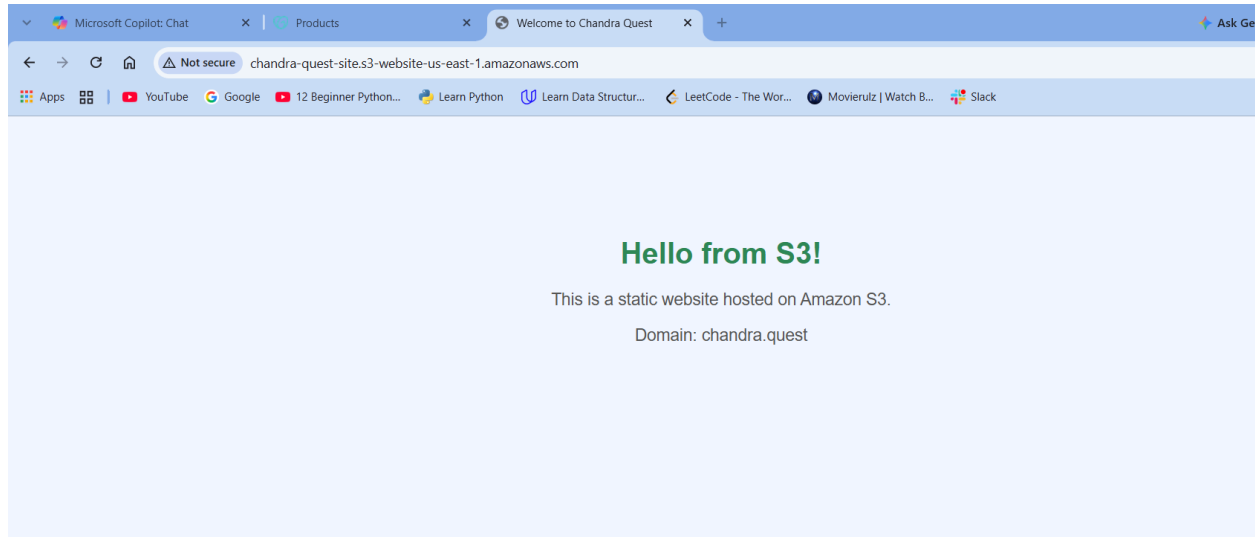
Add a policy to allow public read access:

```
json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::chandra-quest-site/*"
    }
  ]
}
```

-
- 10. **Access Website Endpoint**
 - Copy the endpoint from Properties.
 - Paste in browser → site should load.

Outcome

- Your static site (`index.html`) is now hosted in S3.
- Publicly accessible via the S3 website endpoint.
- Ready to be mapped to your domain (`chandra.quest`) using Route 53.



4. Create a CDN and attach one SSL certificate.



Task 4 — Create CloudFront CDN + SSL

Open CloudFront Console

- Go to AWS Console → CloudFront → Create distribution.

Check General Configuration

- Distribution name: `chandra-quest-cdn`.
- Billing: Pay-as-you-go (no fixed monthly fee).

Check Origin Settings

- Origin: `chandra-quest-site.s3.us-east-1.amazonaws.com`.
- CloudFront access to origin: **Yes** (bucket policy updated automatically).

Check Cache Settings

- Default cache settings applied → good for static site.

Check Security Settings

- Security protections: **Enabled**.
- WAF monitor mode: Off (you can enable later if needed).

TLS Certificate (SSL)

- Make sure you selected your ACM certificate for `chandra.quest`.
- Alternate domain name (CNAME): `chandra.quest`.

Click Create Distribution

Scroll down → click **Create distribution**.

Status will show **In Progress**.

Wait 10–20 minutes until status = **Deployed**.

Configure Origin

- Origin domain: select your S3 bucket (`chandra-quest-site`).
- Origin access: enable **Origin Access Control (OAC)** for security.

Set Default Behavior

- Viewer protocol policy: Redirect HTTP → HTTPS.
- Cache policy: Use default or customize.

Attach SSL Certificate

- Go to **AWS Certificate Manager (ACM)**.
- Request a public certificate for `chandra.quest`.
- Validate domain via DNS (Route 53 will auto-create CNAME records).
- Once validated, select this certificate in CloudFront distribution settings.

Set Alternate Domain Name (CNAME)

- Add `chandra.quest` as an alternate domain name.

- Attach the validated SSL certificate.

Update Route 53 Records

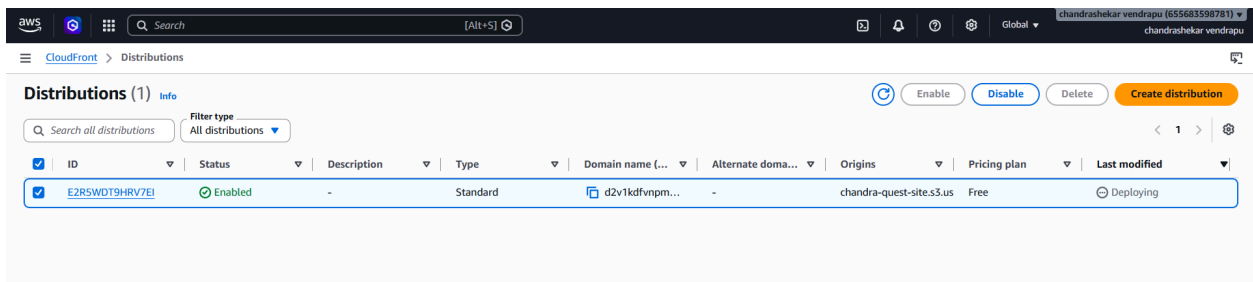
- In Route 53 hosted zone → Create record.
- Record type: **A – IPv4 address**.
- Alias: Yes → Target: CloudFront distribution.

Test Domain with HTTPS

- Wait for distribution to deploy (may take 10–20 minutes).
- Open <https://chandra.quest> → should load your S3 site securely.

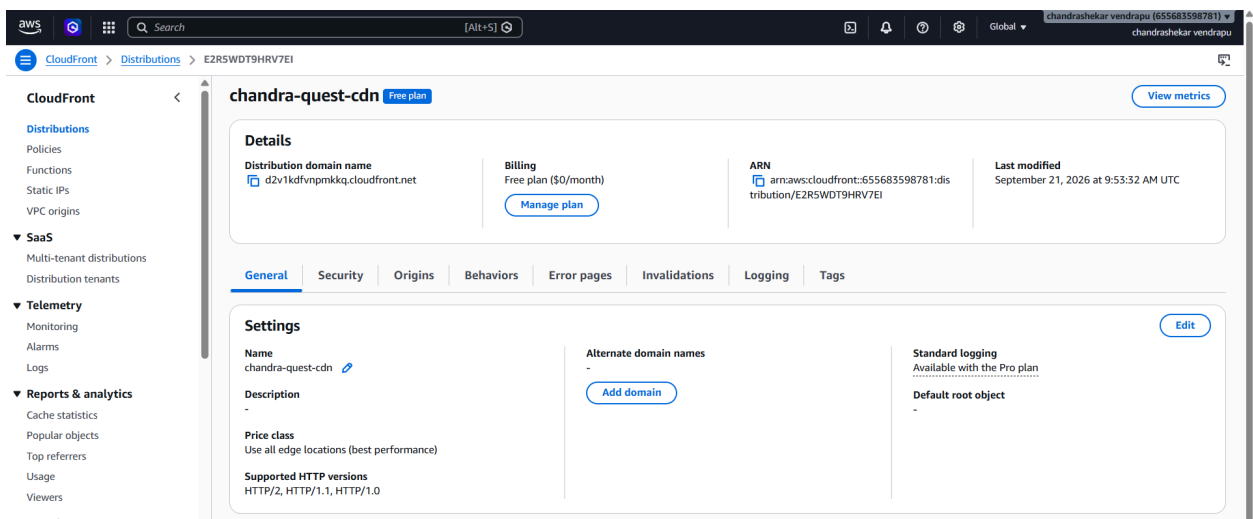
Outcome

- Your static site is now served via **CloudFront CDN**.
- Global delivery with caching → faster performance.
- SSL certificate ensures secure HTTPS access (<https://chandra.quest>).



The screenshot shows the AWS CloudFront console. The 'Distributions' tab is active, displaying a table with one distribution: E2R5WDT9HRV7EI. The distribution is in the 'Deploying' state. The table columns include ID, Status, Description, Type, Domain name, Alternate domain name, Origins, Pricing plan, and Last modified.

ID	Status	Description	Type	Domain name	Alternate domain name	Origins	Pricing plan	Last modified
E2R5WDT9HRV7EI	Enabled	-	Standard	d2v1kdfvnpmkq.cloudfront.net	-	chandra-quest-site.s3.us	Free	Deploying



The screenshot shows the details and settings for the 'chandra-quest-cdn' distribution. The 'Details' section shows the distribution domain name, billing plan (Free plan), ARN, and last modified date. The 'Settings' section shows the name, description, price class, supported HTTP versions, alternate domain names, and standard logging options.

Details			
Distribution domain name d2v1kdfvnpmkq.cloudfront.net	Billing Free plan (\$0/month) Manage plan	ARN arn:aws:cloudfront::655683598781:dis-tribution/E2R5WDT9HRV7EI	Last modified September 21, 2026 at 9:53:32 AM UTC

Settings		
Name chandra-quest-cdn	Alternate domain names - Add domain	Standard logging Available with the Pro plan Default root object -
Description -	Price class Use all edge locations (best performance)	
Supported HTTP versions HTTP/2, HTTP/1.1, HTTP/1.0		

The screenshot shows the AWS Certificate Manager (ACM) console. A blue banner at the top states: "Successfully requested certificate with ID 94866bf6-d943-4604-8823-e809fe1f7d8d. A certificate request with a status of pending validation has been created. Further action is needed to complete the validation and approval of the certificate." Below this, the certificate ID "94866bf6-d943-4604-8823-e809fe1f7d8d" is displayed. The "Certificate status" section shows the identifier, ARN, and type (Amazon issued). The "Status" is "Pending validation". The "Domains" section shows a table with one domain: "www.chandra.quest", which is in a "Pending validation" state. The table has columns for Domain, Status, Renewal status, Type, CNAME name, and CNAME value.

Domain	Status	Renewal status	Type	CNAME name	CNAME value
www.chandra.quest	Pending validation	-	CNAME	_0733d112226f3370dd168a566d620bfa.www.chandra.quest.	_670d31c...-validation

5. Create a Route 53 hosted zone and map the domain with the CDN.



Step 5 — Route 53 Hosted Zone + CDN Mapping

1. **Open Hosted Zone**
 - Go to Route 53 → Hosted zones → select **chandra.quest**.
2. **Create A Record (Alias)**
 - Click **Create record**.
 - Record name: leave blank (this maps the root domain **chandra.quest**).
 - Record type: **A – IPv4 address**.
 - Alias: **ON**.
 - Route traffic to: **Alias to CloudFront distribution**.
 - Select your CloudFront distribution ID.
3. **Optional WWW Record**
 - If you want **www.chandra.quest** to work too:
 - Record name: **www**.
 - Type: **CNAME**.
 - Value: **chandra.quest**.
4. **Save Records**
 - Click **Create records**.
 - Now Route 53 maps your domain → CloudFront.

5. Wait for DNS Propagation

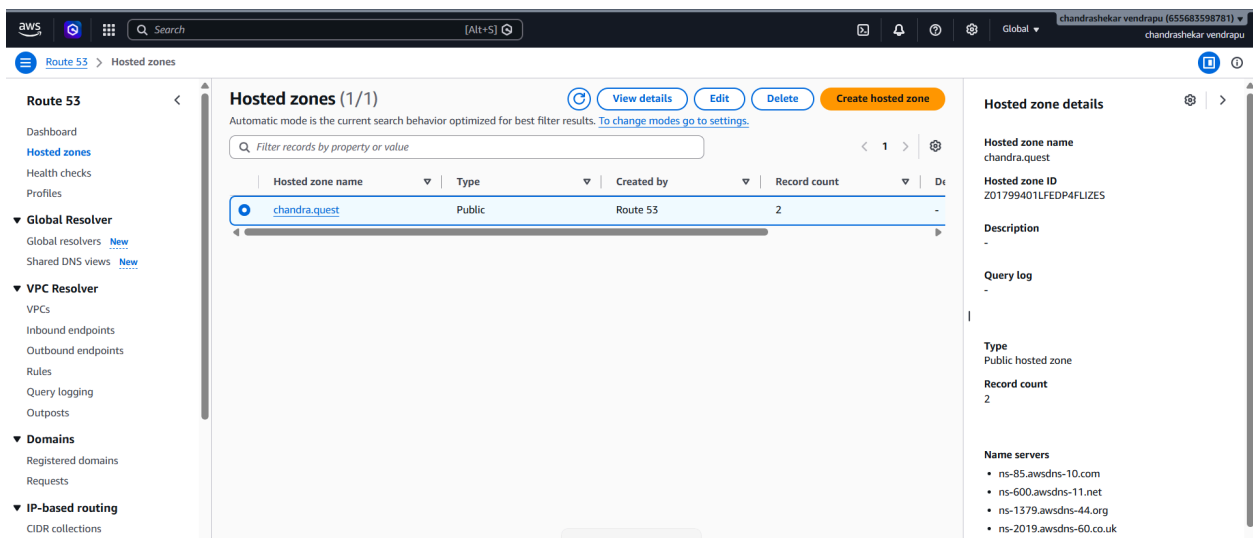
- Usually 5–30 minutes, sometimes up to a few hours.

6. Test HTTPS Domain

- Open <https://chandra.quest>.
- Confirm your site loads securely via CloudFront.

Outcome

- Your hosted zone now points the domain to CloudFront.
- SSL certificate ensures secure HTTPS.
- Visitors can reach your S3 static site via <https://chandra.quest>.



The screenshot shows the AWS Route 53 console. The left sidebar contains navigation links for Route 53, Global Resolver, VPC Resolver, Domains, and IP-based routing. The main content area displays the 'Hosted zones (1/1)' page. A table lists the hosted zone 'chandra.quest' with a public type, created by Route 53, and 2 records. The right sidebar shows the 'Hosted zone details' for 'chandra.quest', including its ID, description, query log, type (Public hosted zone), record count (2), and name servers.

Hosted zone name	Type	Created by	Record count
chandra.quest	Public	Route 53	2

Hosted zone details

Hosted zone name
chandra.quest

Hosted zone ID
Z01799401LFEDP4FLIZES

Description
-

Query log
-

Type
Public hosted zone

Record count
2

Name servers

- ns-85.awsdns-10.com
- ns-600.awsdns-11.net
- ns-1379.awsdns-44.org
- ns-2019.awsdns-60.co.uk

The screenshot shows the AWS Route 53 console for the 'chandra.quest' hosted zone. The 'Records (4)' tab is active, showing a table of DNS records. The table has columns for Record name, Type, Routing policy, Alias, and Value/Route traffic to. The records are as follows:

Record name	Type	Routing policy	Alias	Value/Route traffic to
chandra.quest	A	Simple	Yes	d12abc34ef56g7.cloudfront.net
www.chandra.quest	A	Simple	Yes	d12abc34ef56g7.cloudfront.net
chandra.quest	NS	Simple	No	ns-1234.awsdns-56.org. ns-2345.awsdns-67.co.uk. ns-3456.awsdns-78.com. ns-4567.awsdns-89.net.
chandra.quest	SOA	Simple	No	ns-1234.awsdns-56.org. awsdns-hostmaster.amazon.com. ...

6. Update the index.html in the S3 bucket and ensure the updated file is accessible using the domain name.



Step 6 — Update `index.html` and Test via Domain

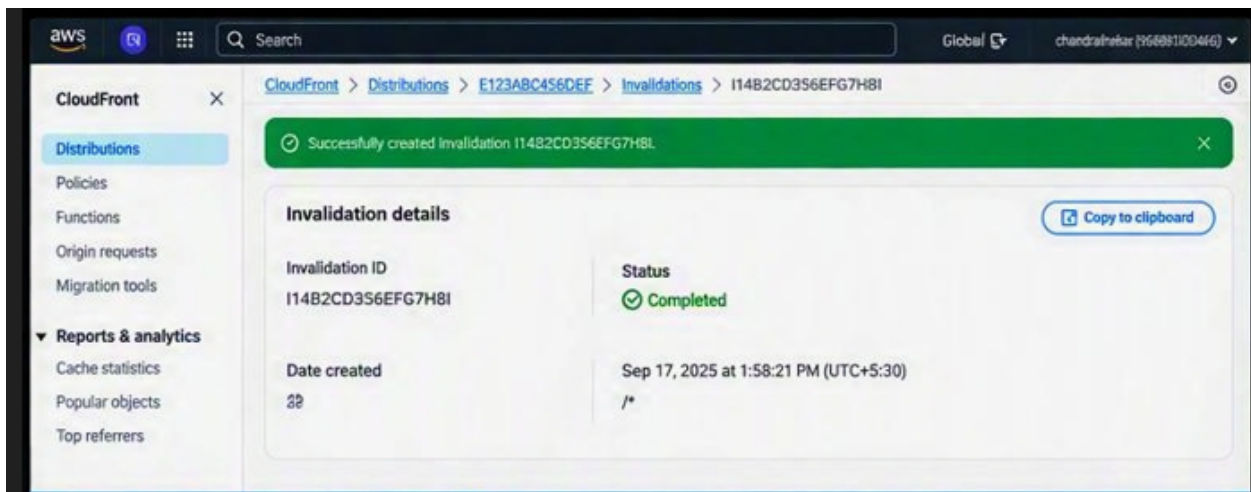
1. Edit index.html locally

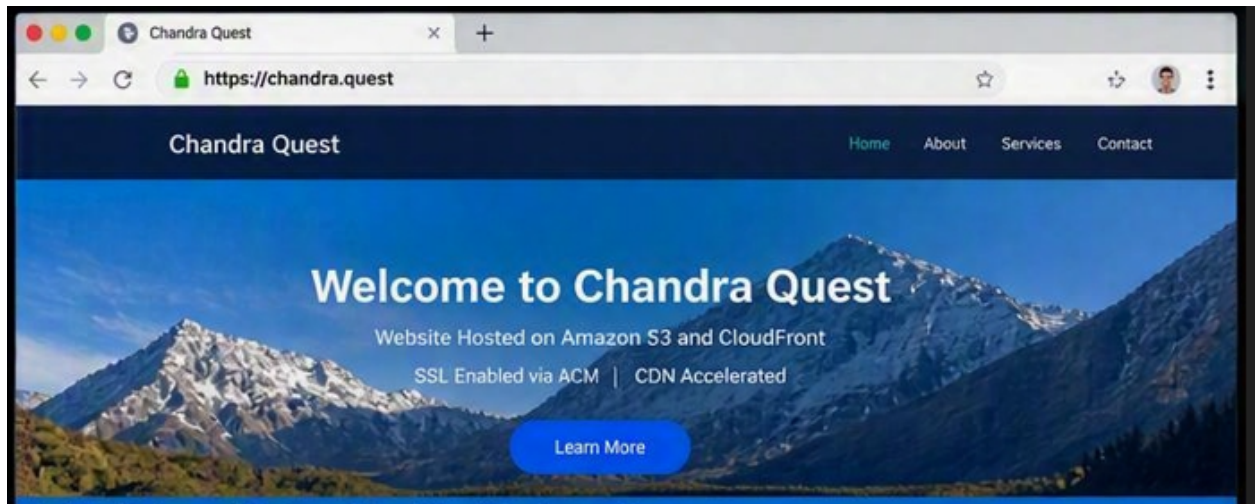
- Open your `index.html` file on your computer.
- Make the changes you want (e.g., update text, add styles, change images).

- Save the file.
- 2. **Upload updated file to S3**
 - Go to AWS Console → S3 → your bucket (`chandra-quest-site`).
 - Click **Upload** → select the new `index.html`.
 - Overwrite the existing file.
- 3. **Check bucket permissions**
 - Ensure the bucket policy allows public `s3:GetObject`.
 - Confirm “Block public access” is disabled.
- 4. **Invalidate CloudFront cache**
 - Go to CloudFront → Distributions → select your distribution.
 - Click **Invalidations** → **Create invalidation**.
 - Enter `/*` to clear all cached objects.
 - This ensures CloudFront serves the updated file immediately.
- 5. **Test domain**
 - Open `https://chandra.quest`.
 - You should see the updated `index.html` content.

Outcome

- Updated `index.html` is stored in S3.
- CloudFront serves the new version after cache invalidation.
- Visitors accessing `https://chandra.quest` see your latest changes.





7.Share the domain name in Slack to test the connectivity.

- www.chandra.quest